

BIG DATA ENGINEERING

Apache Spark & Hadoop Interview Prep Guide

15 core interview topics on Spark internals, architecture, execution model, and the Hadoop MapReduce foundation — compiled into one reference.

Narrow vs. Wide Transformations · Spark Architecture · Unified Engine
· RDD / DataFrame / Dataset · repartition() vs coalesce() ·
Transformations vs Actions · Driver vs Executor · Jobs / Stages / Tasks
· Lazy Evaluation · Spark Query Plan · DAG · Hadoop MapReduce ·
Spark Components · Job vs Stage vs Task · SparkSession vs
SparkContext

Table of Contents

1. Narrow vs. Wide Transformations
2. Apache Spark Architecture
3. Apache Spark Is a Unified Engine
4. RDD vs. DataFrame vs. Dataset
5. repartition() vs. coalesce()
6. Transformations vs. Actions
7. Driver vs. Executor
8. Jobs, Stages, and Tasks
9. Lazy Evaluation
10. Spark Query Plan
11. What Is a DAG?
12. Hadoop MapReduce
13. Apache Spark Components
14. Job vs. Stage vs. Task (Quick Comparison)
15. SparkSession vs. SparkContext

Narrow vs. Wide Transformations

Understanding narrow and wide transformations is one of the most important Spark interview topics because it explains how Spark executes your code.

What Is a Transformation?

A transformation is an operation that creates a new RDD or DataFrame from an existing one. Examples: `df.filter(...)`, `df.select(...)`, `df.join(...)`, `df.groupBy(...)`.

Transformations are **lazy** — Spark does not execute them immediately. It builds an execution plan (DAG) and runs it only when an action (like `show()` or `count()`) is called.

Narrow Transformation

DEFINITION

Each output partition depends on only **one** input partition. No data needs to move between executors.

Input Partitions

```
P1 → P1
P2 → P2
P3 → P3
P4 → P4
```

Each partition is processed independently.

Example: `df.filter(df.age > 18)`

Partition	Before	After filter
P1	Ali 20, Sara 15	Ali 20
P2	John 30, Omar 17	John 30

Each partition filters its own data — no communication with other partitions required.

Common Narrow Transformations

- `map()` — `rdd.map(lambda x: x * 2)`
- `filter()` — `df.filter(df.salary > 5000)`
- `select()` — `df.select("name", "salary")`
- `withColumn()` — `df.withColumn("tax", df.salary * 0.1)`
- `drop()` — `df.drop("address")`
- `union()` — `df1.union(df2)` (when no shuffle is required)
- `coalesce()` — `df.coalesce(4)` (usually narrow — merges partitions without a full shuffle)

Characteristics of Narrow Transformations

No shuffle

Faster

Less network traffic

Less disk I/O

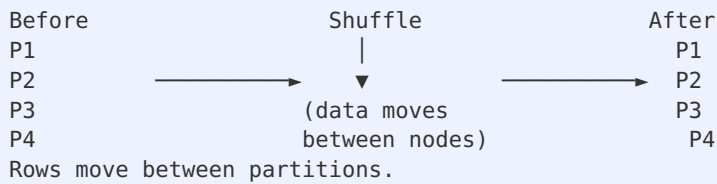
Independent partitions

Same stage

Wide Transformation

DEFINITION

An output partition depends on **multiple** input partitions. Spark must shuffle data across the cluster.



Example: `df.groupBy("country").sum("sales")` — Spark cannot compute totals independently because all "Egypt" rows must be grouped together, so it shuffles the data first, then aggregates.

Common Wide Transformations

- `groupBy()` — `df.groupBy("country").count()`
- `join()` — `customers.join(orders, "customer_id")`
- `distinct()` — `df.distinct()`
- `dropDuplicates()` — `df.dropDuplicates()`
- `orderBy()` / `sort()` — `df.orderBy("salary")`
- `repartition()` — `df.repartition(10)` (always shuffles)
- `reduceByKey()` / `aggregateByKey()` (RDD)

Characteristics of Wide Transformations

Shuffle required

Network communication

More disk I/O

Slower

New stage

Narrow vs. Wide — Performance Comparison

Feature	Narrow	Wide
Shuffle	No	Yes
Network transfer	No	Yes
Speed	Faster	Slower
New stage	Usually no	Yes
Parallelism	High	Depends on shuffle
Resource usage	Lower	Higher

Interview Examples

Narrow chain (same stage, no shuffle):

```
df.filter(df.age > 20) \
  .select("name", "salary") \
  .withColumn("tax", df.salary * 0.1)
```

Wide chain (introduces a shuffle → new stage):

```
df.filter(df.age > 20) \
  .groupBy("country") \
  .sum("salary")
```

Filter → (No shuffle) → GroupBy → Shuffle → Aggregation
The groupBy() introduces a shuffle, which ends one stage and starts another.

Q: Why are wide transformations slower than narrow transformations?

Because wide transformations require a shuffle, where Spark redistributes data across partitions and executors. Shuffling involves network communication, disk I/O, and synchronization — one of the most expensive operations in Spark.

Summary Table

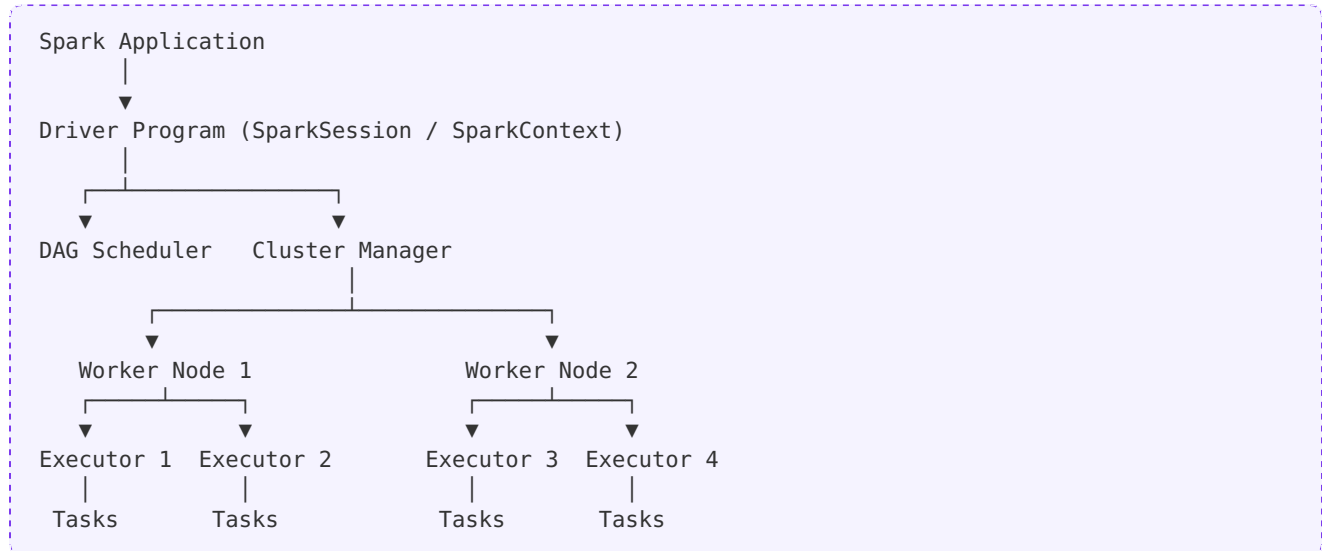
Narrow Transformations	Wide Transformations
map()	groupBy()
filter()	join()
select()	orderBy() / sort()
withColumn()	distinct()
drop()	dropDuplicates()
flatMap()	repartition()
mapPartitions()	reduceByKey() (RDD)
coalesce() (default)	aggregateByKey() (RDD)

RULE TO REMEMBER

- **Narrow:** each output partition depends on one input partition → no shuffle needed.
- **Wide:** an output partition depends on multiple input partitions → a shuffle is required.

Apache Spark Architecture

Apache Spark follows a **master-worker architecture** for distributed data processing, with five main components: Driver, Cluster Manager, Worker Nodes, Executors, and Tasks.



1. Driver Program

The Driver is the **brain** of a Spark application. It is responsible for:

- Creating the SparkSession (and internally the SparkContext)
- Reading your Spark code and building the execution plan
- Optimizing the plan (using Catalyst for DataFrame/SQL operations)
- Creating the DAG (Directed Acyclic Graph)
- Dividing the DAG into stages, and each stage into tasks
- Sending tasks to executors and monitoring execution
- Collecting results

The Driver runs only **one instance** per Spark application.

2. Cluster Manager

Allocates resources for Spark applications: how many executors to start, where they run, and CPU/memory allocation. Spark supports Standalone, Apache YARN, Kubernetes, and Apache Mesos. The Cluster Manager does **not** execute tasks — it only manages resources.

3. Worker Nodes

The machines providing computing resources. Each worker node can run one or more executors, hosting them and providing CPU/memory for assigned tasks.

4. Executors

Executors are JVM processes launched on worker nodes for a Spark application. They execute tasks, read data from storage, perform transformations/actions, cache data, write results, and report status to the Driver. Executors exist only for the lifetime of the application.

5. Tasks

A Task is the smallest unit of work in Spark — each task processes one data partition. A DataFrame with 8 partitions creates 8 tasks for a stage.

How Spark Executes a Job

```
df = spark.read.csv("employees.csv", header=True)
df.filter(df.salary > 5000) \
  .groupBy("department") \
  .count() \
  .show()
```

User Program → SparkSession → Driver → Logical Plan → Optimized Plan
→ Physical Plan → DAG → Stages → Tasks → Executors → Results

1. **SparkSession** starts the application.
2. **Driver** reads code, builds and optimizes the logical plan, creates a physical plan and DAG.
3. **DAG → Stages**: narrow transformations (e.g. filter) stay in the same stage; wide transformations (e.g. groupBy, join) require a shuffle and start a new stage.
4. **Stages → Tasks**: each stage splits into tasks based on partition count (e.g. 200 partitions → 200 tasks).
5. **Cluster Manager** launches executors on worker nodes.
6. **Executors** process data in parallel and return results.
7. **Driver** collects the final output.

Real-World Example

A company has 1 TB of sales data in HDFS. Instead of one machine processing it all: the file is divided into many partitions, Spark creates one task per partition, tasks are distributed across executors on multiple worker nodes, and the Driver combines the results into the final output.

Component Summary

Component	Responsibility
Driver	Creates execution plan, schedules jobs, monitors execution, collects results
Cluster Manager	Allocates cluster resources and launches executors
Worker Node	Provides CPU and memory resources; hosts executors

Executor	Executes tasks, caches data, reports status to the Driver
Task	Smallest unit of work; processes one data partition

Interview Q&A

Q1. What are the main components of Spark architecture?

Driver, Cluster Manager, Worker Nodes, Executors, Tasks.

Q2. Why is the Driver called the brain of Spark?

It creates the execution plan, optimizes queries, builds the DAG, divides work into stages and tasks, schedules execution, and monitors the whole application.

Q3. Difference between a Worker Node and an Executor?

A Worker Node is a machine providing CPU/memory resources. An Executor is a JVM process running on a worker node that executes tasks for a specific Spark application. A worker node can host multiple executors.

Q4. What is the smallest unit of execution?

A Task — each task processes one data partition.

Q5. Role of the Cluster Manager?

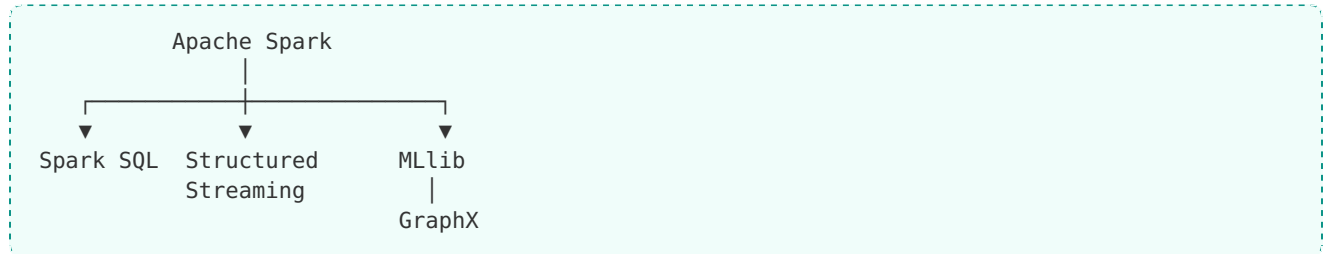
Allocates resources, launches executors, and manages CPU/memory across the cluster. It does not execute tasks itself.

INTERVIEW SUMMARY

- Spark uses a master-worker architecture for distributed processing.
- The Driver is the brain: builds the DAG, splits it into stages/tasks, schedules execution, collects results.
- The Cluster Manager allocates resources and launches executors on worker nodes.
- Worker Nodes provide computing resources and host executors.
- Executors execute tasks, cache data, and report progress to the Driver.
- A Task is the smallest unit of work and processes exactly one partition.

Apache Spark Is a Unified Engine

Apache Spark is called a **unified analytics engine** because it provides one engine that can handle many different types of data processing without requiring separate frameworks.



Why Is Spark Called a Unified Engine?

Because all components share the same:

- Execution engine (the same engine executes everything)
- Cluster manager (Standalone, YARN, Kubernetes, Mesos)
- Programming APIs (Python, Scala, Java, R)
- Data abstractions (RDDs, DataFrames, Datasets)
- Optimization engine (Catalyst + Tungsten for SQL/DataFrames)

This means you don't need different systems for SQL, streaming, and machine learning.

Example: Fraud Detection Pipeline

1. Read transaction data from Kafka using Structured Streaming
2. Clean and transform the data using DataFrames
3. Run SQL queries with Spark SQL
4. Predict fraud using an MLlib model
5. Save results to a data lake

All of this runs inside **one** Spark application, using the same execution engine.

Before Spark — Separate Engines

SQL Processing → Hive
 Streaming → Storm
 Machine Learning → Mahout
 Graph Processing → Giraph
 Each tool had its own APIs, execution engine, and deployment model.

With Spark — One Unified Engine



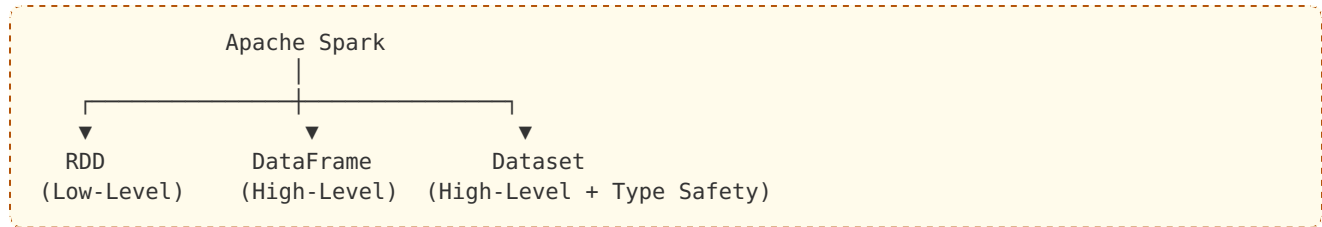
▼ ▼ ▼ ▼
SQL Streaming MLlib GraphX
Everything runs on the same engine.

INTERVIEW ANSWER

Apache Spark is a unified analytics engine because it provides a single execution engine that supports batch processing, SQL queries, real-time stream processing, machine learning, and graph processing through one programming model and one cluster management system.

RDD vs. DataFrame vs. Dataset

Apache Spark provides three main data abstractions:



1. RDD (Resilient Distributed Dataset)

Spark's original distributed data structure — an immutable, fault-tolerant, distributed collection of objects, offering low-level control over distributed computations.

Characteristics: low-level API · stores any object type · immutable · distributed across partitions · fault tolerant via lineage · no schema · no Catalyst Optimizer · no Tungsten optimization.

```
rdd = spark.sparkContext.parallelize([1, 2, 3, 4])
result = rdd.map(lambda x: x * 2)
print(result.collect())
```

Advantages: fine-grained control, suitable for complex custom transformations, works with unstructured data.

Disadvantages: slower than DataFrames/Datasets, no automatic optimization, no schema, more code.

2. DataFrame

A distributed collection of data organized into named columns, similar to a relational table or Pandas DataFrame. Internally, a DataFrame is a *Dataset of Rows*.

Characteristics: high-level API · has a schema · uses Catalyst Optimizer · uses Tungsten execution engine · much faster than RDD · supports SQL queries.

```
df = spark.read.csv("employees.csv", header=True)
df.filter(df.salary > 5000).show()
```

Advantages: very fast, easy to use, SQL support, automatic optimization, less memory, preferred for ETL/analytics.

Disadvantages: rows are not strongly typed, less low-level control than RDD.

3. Dataset

Combines the advantages of RDDs and DataFrames: strong compile-time type safety, Catalyst optimization, Tungsten execution, and object-oriented programming support.

IMPORTANT

The Dataset API is available only in Scala and Java — **PySpark does not support typed Datasets.**

```
case class Employee(name: String, salary: Double)
val ds = Seq(
  Employee("Ali", 5000),
  Employee("Sara", 7000)
).toDS()
```

Advantages: strongly typed, compile-time error checking, high performance, Catalyst optimization, OOP.

Disadvantages: not available in Python, slightly more complex than DataFrames.

Performance Comparison

Fastest → DataFrame ≈ Dataset Slowest → RDD

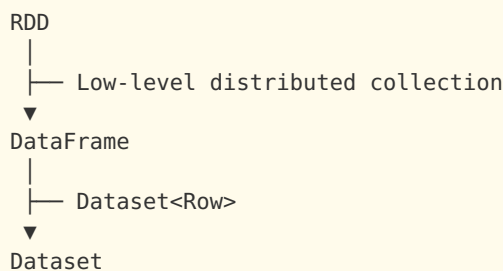
Feature Comparison

Feature	RDD	DataFrame	Dataset
API Level	Low	High	High
Schema	No	Yes	Yes
Type Safety	No	No (Rows)	Yes
Catalyst Optimizer	No	Yes	Yes
Tungsten Engine	No	Yes	Yes
SQL Support	No	Yes	Yes
Compile-time Type Checking	No	No	Yes
Performance	Good	Excellent	Excellent
Languages	Scala, Java, Python	Scala, Java, Python, R	Scala, Java

When Should You Use Each?

- **RDD:** low-level transformations, unstructured data, custom distributed algorithms, direct control over partitions.
- **DataFrame:** ETL pipelines, structured/semi-structured data, SQL queries, joins/aggregations, PySpark — the most common choice for Data Engineering.
- **Dataset:** Scala/Java, compile-time type safety, OOP with Spark, DataFrame performance with typed objects.

Relationship Between Them



Interview Q&A

Q1. Which is fastest: RDD, DataFrame, or Dataset?

DataFrame and Dataset are generally faster than RDD because Spark optimizes them via Catalyst and Tungsten.

Q2. Why is DataFrame faster than RDD?

Schema information, Catalyst Optimizer, Tungsten execution engine, better memory management, optimized query execution.

Q3. Can you use SQL with an RDD?

No — SQL works with DataFrames and Datasets, not directly with RDDs.

Q4. Does PySpark support Dataset?

No. PySpark supports RDD and DataFrame; typed Dataset is Scala/Java only.

Q5. Most common abstraction in Data Engineering?

DataFrame — high performance, SQL support, automatic optimization, ease of use.

INTERVIEW SUMMARY

- RDD: original low-level abstraction — immutable, fault-tolerant, fine-grained control, no schema/optimization.
- DataFrame: high-level distributed table — SQL support, Catalyst + Tungsten, most used API in modern Spark (especially PySpark).
- Dataset: DataFrame performance + compile-time type safety, but Scala/Java only.
- Use DataFrames for almost all ETL/analytics/SQL; RDDs for low-level control; Datasets for typed Scala/Java work.

repartition() vs. coalesce()

Both `repartition()` and `coalesce()` change the number of partitions in a DataFrame or RDD, but they work very differently.

What Is a Partition?

A logical chunk of data — Spark processes one partition per task. Changing the number of partitions affects parallelism, performance, and resource utilization.

repartition()

Redistributes data across **all** partitions to create a new number of partitions. Performs a **full shuffle** — data moves across the cluster.

- Can increase or decrease partitions
- Always performs a shuffle
- Produces evenly balanced partitions

```
df2 = df.repartition(8) # e.g. 4 partitions → 8 balanced partitions
```

coalesce()

Primarily used to **reduce** partitions. Tries to merge existing partitions without a full shuffle.

- Primarily decreases partitions
- Avoids a full shuffle when reducing
- Faster than `repartition()` for reducing
- May create uneven partition sizes

```
df2 = df.coalesce(2) # e.g. P1,P2,P3,P4 → P1+P2, P3+P4 (merged, little movement)
```

Shuffle Behavior

Operation	Shuffle?
<code>repartition()</code>	Yes (always)
<code>coalesce()</code>	Usually no (when reducing partitions)

A shuffle is expensive because it transfers data across the network and writes intermediate data.

Increasing Partitions

repartition() supports increasing partitions: `df.repartition(20)`. **coalesce()** technically accepts a larger number but is not intended for increasing partitions since it doesn't redistribute data effectively — use `repartition()` instead.

Typical Use Cases

Use repartition() when...	Use coalesce() when...
Increasing the number of partitions	Reducing partitions
You need evenly distributed data	Writing output files
Preparing data before a large join/aggregation	Avoiding unnecessary shuffles
Avoiding data skew	Improving performance with fewer partitions

```
# Write a single output file (fine for small data, not for very large datasets)
df.coalesce(1).write.mode("overwrite").csv("output")
```

Comparison Table

Feature	repartition()	coalesce()
Purpose	Increase or decrease partitions	Primarily decrease partitions
Shuffle	Always	Usually no (when reducing)
Performance	Slower	Faster
Data Distribution	Evenly balanced	May be uneven
Increase Partitions	Yes	Not recommended
Decrease Partitions	Yes	Yes
Typical Use	Rebalance data, increase parallelism	Reduce partitions before writing

Interview Q&A

Q1. Difference between repartition() and coalesce()?

`repartition()` changes partitions via a full shuffle, resulting in evenly distributed data. `coalesce()` primarily reduces partitions by merging existing ones, usually without a full shuffle — faster.

Q2. Which is faster?

`coalesce()` is generally faster when reducing partitions since it avoids a full shuffle.

Q3. Which can increase partitions?

`repartition()`. `coalesce()` is not designed for increasing partitions.

Q4. Why is `repartition()` more expensive?

It performs a full shuffle, redistributing data across executors to create balanced partitions.

Q5. When should you use `coalesce(1)`?

Typically before writing a small dataset when you need a single output file. Avoid for very large datasets — it reduces parallelism and can bottleneck performance.

INTERVIEW SUMMARY

- Both change the number of partitions in Spark.
- `repartition()` can increase or decrease partitions and always performs a full shuffle → balanced partitions.
- `coalesce()` mainly reduces partitions and usually avoids a full shuffle → more efficient for that purpose.
- Use `repartition()` for load balancing / more parallelism; use `coalesce()` when reducing partitions, especially before writing output.

Transformations vs. Actions

Spark operations fall into two categories. **Transformations** create a new DataFrame or RDD without executing the computation immediately. **Actions** trigger execution of all pending transformations and either return a result or write data to storage.

What Is a Transformation?

Creates a new DataFrame/RDD from an existing one. Lazy — Spark records the operations in a lineage (execution plan) instead of executing immediately.

Characteristics: lazy evaluation · returns a new DataFrame/RDD · does not execute immediately · builds the logical DAG · chainable.

```
df_filtered = df.filter(df.salary > 5000)
df_selected = df_filtered.select("name", "salary")
# No data has been read or processed yet – only recorded.
```

What Is an Action?

Triggers execution of all previously defined transformations, returning a result to the Driver or saving data to storage.

Characteristics: triggers job execution · produces a result · starts DAG execution · causes Spark to read/process data.

```
df_filtered.show()
# Spark executes all pending transformations, reads data,
# processes across executors, returns first rows to the Driver.
```

Example Workflow

```
df = spark.read.csv("employees.csv", header=True) # Transformation
df2 = df.filter(df.salary > 5000)                 # Transformation
df3 = df2.select("name", "salary")                 # Transformation
df3.show()                                         # Action → execution starts
```

Lazy Evaluation

Spark records all transformations, builds a logical plan, optimizes it, and executes everything only when an action is called — reducing unnecessary work and improving performance.

Common Transformations

DataFrame: select() · filter()/where() · withColumn() · drop() · dropDuplicates() · distinct() · groupBy() · orderBy()/sort() · join() · union() · limit() · repartition() · coalesce()

RDD: map() · flatMap() · filter() · distinct() · union() · intersection() · reduceByKey() · groupByKey() · sortByKey()

Common Actions

DataFrame: show() · collect() · count() · first() · head() · take() · foreach() · write.save()/csv()/parquet()/json()

RDD: collect() · count() · first() · take() · reduce() · foreach() · saveAsTextFile()

Comparison Table

Feature	Transformation	Action
Purpose	Define how data should be processed	Execute the processing
Execution	Lazy (not executed immediately)	Immediate (triggers execution)
Returns	New DataFrame or RDD	Result to Driver, or writes data
Builds DAG	Yes	Executes the DAG
Creates Spark Job	No	Yes

Why Does Spark Use Lazy Evaluation?

To optimize the execution plan, combine multiple transformations into a single optimized pipeline, minimize data movement (shuffle), avoid unnecessary computations, and improve overall performance.

Interview Q&A

Q1. Difference between a transformation and an action?

Transformation: defines a new DataFrame/RDD, evaluated lazily, doesn't execute immediately. Action: triggers execution of all pending transformations and returns a result or writes data.

Q2. Does a transformation execute immediately?

No — transformations are lazy; Spark records them and waits until an action is called.

Q3. Which operation starts a Spark job?

An Action starts a Spark job by triggering DAG execution.

Q4. Give examples of transformations.

filter(), select(), join(), groupBy(), withColumn(), repartition(), map() (RDD).

Q5. Give examples of actions.

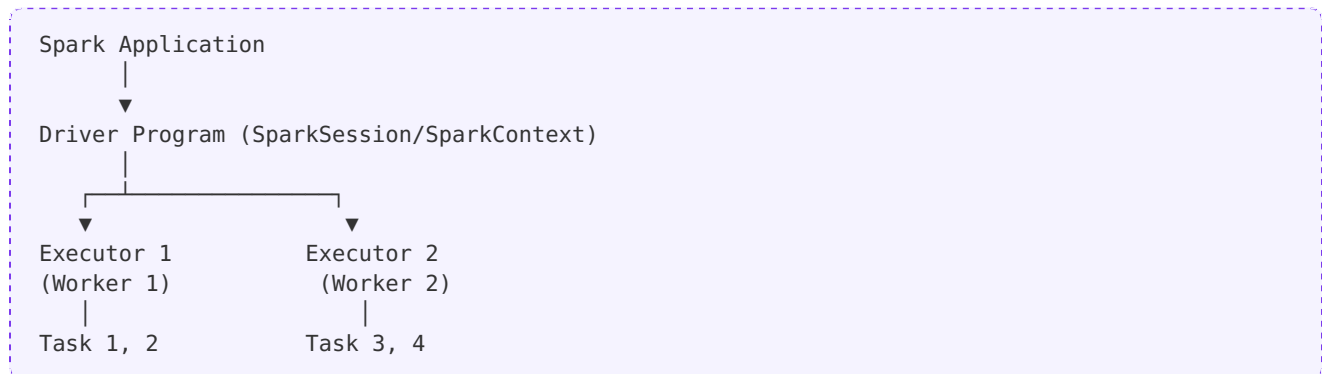
`show()`, `count()`, `collect()`, `first()`, `take()`, `write.parquet()`, `saveAsTextFile()`.

INTERVIEW SUMMARY

- Transformations define how data should be processed; lazy, return a new DataFrame/ RDD, build the execution plan.
- Actions trigger execution of pending transformations — return results (`count()`, `collect()`, `show()`) or write data (`write.parquet()`).
- Lazy evaluation lets Catalyst and the scheduler optimize the plan before running it.

Driver vs. Executor

The Driver and Executors are the two most important components in Spark architecture. The **Driver** is the brain — it plans, coordinates, and monitors execution. **Executors** are the workers that perform the actual data processing.



What Is the Driver?

The central coordinator of a Spark application — one Driver per application. Responsibilities:

- Creates the SparkSession / SparkContext
- Reads and understands your Spark code; builds and optimizes the logical plan
- Creates the DAG, splits it into stages, splits stages into tasks (one per partition)
- Requests resources from the Cluster Manager and sends tasks to executors
- Monitors execution, handles failures/retries, collects results

What Is an Executor?

A JVM process running on a Worker Node that performs the actual computations. Each application gets its own executors, terminated when the application finishes.

- Executes tasks assigned by the Driver
- Reads data from HDFS, S3, databases, etc.
- Performs transformations and actions; caches data in memory
- Writes output; reports task status/results back to the Driver

How They Work Together

1. Driver reads code → 2. Builds Logical Plan → 3. Optimizes plan
- 4. Creates DAG → 5. Splits into Stages → 6. Creates Tasks
- 7. Sends Tasks to Executors → 8. Executors process in parallel
- 9. Executors return results → 10. Driver returns final output

Driver vs. Executor

Feature	Driver	Executor
Role	Coordinates the application	Performs computations
Runs On	Driver node	Worker nodes
Number	One per application	One or more per application
Creates SparkSession	Yes	No
Creates DAG / Stages / Tasks	Yes	No
Executes Tasks	No	Yes
Reads Data	Coordinates reading	Reads assigned partitions
Caches Data	No	Yes
Returns Results	Receives results	Sends results to Driver

Interview Q&A

Q1. What is the Driver in Spark?

The central coordinator: creates the execution plan, builds the DAG, divides it into stages/tasks, schedules work, monitors execution, and collects results.

Q2. What is an Executor?

A JVM process on a worker node that executes tasks, processes data partitions, caches data in memory, and reports results back to the Driver.

Q3. Can a Spark application have multiple Drivers?

No — one Driver, but it can have multiple Executors.

Q4. Who creates Tasks?

The Driver, after dividing the DAG into stages.

Q5. Who executes Tasks?

The Executors, as assigned by the Driver.

Q6. Does the Driver process data?

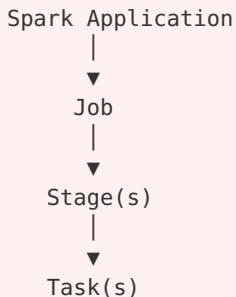
Its primary role is coordination, not large-scale processing — Executors process the data. However, actions like `collect()` or `take()` return data to the Driver; using `collect()` on very large datasets can cause it to run out of memory.

INTERVIEW SUMMARY

- The Driver is the brain: creates the SparkSession, builds/optimizes the plan, creates the DAG, divides it into stages and tasks, schedules, monitors, and collects results.
- Executors are JVM processes on worker nodes: execute tasks, cache intermediate data, write output, report status.
- One Driver, one or more Executors per application. Driver plans and coordinates; Executors compute in parallel.

Jobs, Stages, and Tasks

Every Spark application executes at three levels:



1. Job

The highest-level unit of execution. A new job is created every time an **action** is called (`show()`, `count()`, `collect()`, `take()`, `first()`, `write.csv()`, `write.parquet()`).

```
df.filter(df.salary > 5000).show()    # → 1 Spark Job

df.count()    # → Job 1
df.show()     # → Job 2 (each action starts a new job)
```

2. Stage

A group of tasks that can execute **without requiring a shuffle**. Spark divides a job into multiple stages whenever it encounters a wide transformation (`groupBy()`, `join()`, `distinct()`, `orderBy()`, `repartition()`) — a shuffle boundary.

```
df.filter(df.salary > 5000) \
  .groupBy("department") \
  .count() \
  .show()
```

```
Stage 1: Read CSV → Filter → [Shuffle]
Stage 2: GroupBy → Count → Show
```

Narrow vs. Wide (Recap)

Narrow (no shuffle, same stage): `filter()`, `select()`, `withColumn()`, `map()`.

Wide (shuffle, new stage): `groupBy()`, `join()`, `distinct()`, `orderBy()`, `repartition()`.

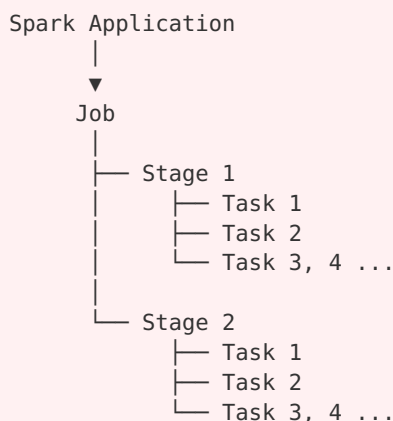
3. Task

The smallest unit of execution — each task processes one data partition. 8 partitions → 8 tasks; 200 partitions → 200 tasks.

Complete Execution Flow

```
graph TD
    Action["Action (show)"] --> Job
    Job --> Stage1["Stage 1: Read CSV → Filter → [Shuffle]"]
    Job --> Stage2["Stage 2: GroupBy → Count → Show"]
    Stage1 --> Tasks["Each Stage → Tasks (one per partition)"]
    Stage2 --> Tasks
```

Relationship Between Job, Stage, and Task



Real-World Example

A company processes 500 GB of sales data across 100 partitions, running `filter → join → groupBy → sum → show`. Spark executes 1 Job (`show()`), multiple Stages (`join()` and `groupBy()` require shuffles), and 100 Tasks per Stage.

Comparison Table

Feature	Job	Stage	Task
Definition	Complete unit of work triggered by an action	Group of operations executed without a shuffle	Smallest unit of execution
Created By	Action (<code>show</code> , <code>count</code> , <code>collect</code>)	Shuffle boundaries (wide transformations)	Data partitions
Execution Level	Highest	Middle	Lowest
Contains	Stages	Tasks	Processes one partition
Runs On	Driver coordinates	Executors execute stages via tasks	Executors

Interview Q&A

Q1. What is a Job in Spark?

The highest-level execution unit — created whenever an action is called; represents the complete work required to produce the result.

Q2. What is a Stage?

A group of tasks that execute without a shuffle. A new stage is created at every wide-transformation shuffle boundary.

Q3. What is a Task?

The smallest unit of execution — processes one data partition and runs on an executor.

Q4. What creates a new Job?

An action, e.g. `show()`, `count()`, `collect()`, `write.parquet()`.

Q5. What creates a new Stage?

A shuffle — usually from `groupBy()`, `join()`, `distinct()`, `orderBy()`, `repartition()`.

Q6. What determines the number of Tasks?

The number of partitions in a stage (e.g. 100 partitions → 100 tasks).

MEMORY SHORTCUT

- **Action** ⇒ Job
- **Shuffle (wide transformation)** ⇒ New Stage
- **Partition** ⇒ Task

Lazy Evaluation in Apache Spark

Lazy Evaluation is one of the core concepts of Spark: Spark does not execute transformations immediately. Instead, it records them, builds an execution plan (DAG), and waits until an **action** is called before executing the computation.

Why Does Spark Use Lazy Evaluation?

- Optimize the execution plan
- Reduce unnecessary computations
- Minimize data shuffling
- Combine multiple transformations into a single optimized execution
- Improve performance and resource utilization

How It Works

```
df = spark.read.csv("employees.csv", header=True)
df2 = df.filter(df.salary > 5000)
df3 = df2.select("name", "salary")
df4 = df3.orderBy("salary")
# At this point: no data read, no filtering, no sorting has happened.

df4.show() # Now Spark builds/optimizes the DAG, creates stages & tasks, executes.
```

Your Code → Transformations (filter, select, orderBy)
→ Spark Records Operations → Builds DAG → Optimizes DAG
→ Action (show/count/collect) → Execution Starts

Lazy Transformations

select() · filter() · where() · withColumn() · drop() · join() · groupBy() · orderBy() · repartition() · coalesce() · map()/flatMap() (RDD)

Actions Trigger Execution

show() · count() · collect() · first() · take() · foreach() · write.csv() · write.parquet()

Benefits of Lazy Evaluation

1. **Query Optimization** — Spark can push a filter closer to the data source and read only required columns, reducing I/O.
2. **Reduced Data Movement** — minimizes unnecessary shuffles.
3. **Fewer Computations** — if an intermediate DataFrame is never used in an action, Spark never computes it.

4. **Better Resource Utilization** — CPU, memory, network used only when actually needed.

Lazy vs. Immediate Execution

Lazy Evaluation (Spark)	Immediate Execution (other systems)
Execution begins only when an action is called	Filter → Execute, Select → Execute, GroupBy → Execute, Count → Execute — can cause unnecessary work

Real-World Example

For a 500 GB sales dataset: read → filter 2025 records → join with customers → group by country → sum sales → show. Spark records all transformations, builds one optimized DAG, and executes the plan only when `show()` is called — avoiding separate disk reads and network transfers per step.

Interview Q&A

Q1. What is Lazy Evaluation?

Spark's execution strategy where transformations aren't executed immediately — Spark records them, builds an optimized DAG, and executes only when an action is called.

Q2. Why does Spark use it?

To optimize execution plans, reduce unnecessary computation, minimize shuffling, improve performance and resource efficiency.

Q3. Which operations are lazy?

Transformations like `filter()`, `select()`, `join()`, `groupBy()`, `map()`.

Q4. Which operations trigger execution?

Actions like `show()`, `count()`, `collect()`, `take()`, `write.parquet()`.

Q5. Does `filter()` execute immediately?

No — it's a transformation; Spark records it and waits for an action.

Q6. Relationship between Lazy Evaluation and DAG?

Laziness lets Spark collect all transformations, build a DAG, optimize it, and then execute efficiently when an action triggers it.

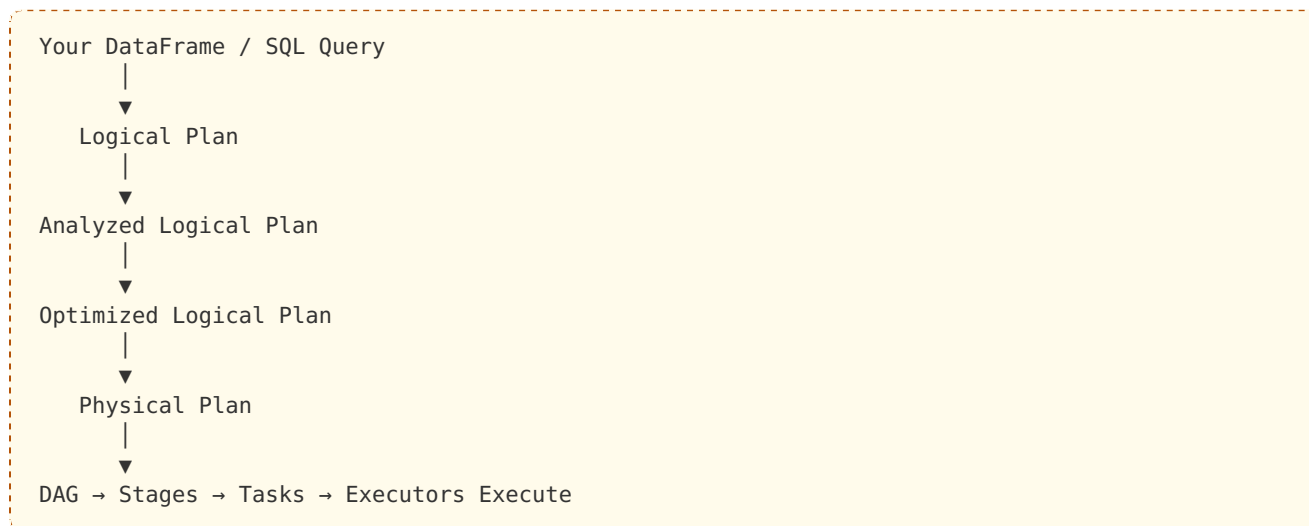
INTERVIEW SUMMARY

- Lazy Evaluation delays execution until an action is called.
- During this time Spark records transformations, builds an optimized DAG, and determines the most efficient execution plan.

- Reduces unnecessary computation, minimizes shuffling, improves resource utilization and performance.
- Transformations are lazy; actions trigger execution.

Spark Query Plan

A Spark Query Plan is the sequence of steps Spark generates to execute your DataFrame or SQL query efficiently. Before processing any data, Spark analyzes, optimizes, and creates an execution plan through four main stages.



1. Logical Plan

Created immediately after you write your code. Describes *what* operations should be performed, without considering *how*. No optimization yet, no data read — part of lazy evaluation.

```
df.filter(df.salary > 5000).select("name", "salary")
```

```

Project [name, salary]
  |
Filter salary > 5000
  |
Scan employees
  
```

2. Analyzed Logical Plan

Spark verifies the query is valid: do tables/columns exist, are data types compatible, are functions valid. If not, Spark throws an `AnalysisException` (e.g. `Column 'income' does not exist`).

3. Optimized Logical Plan

The **Catalyst Optimizer** improves the query without changing its result:

- **Filter Pushdown** — push the filter to the data source, reading only matching rows (reduces disk I/O).
- **Column Pruning** — read only the required columns instead of every column.

- **Constant Folding** — evaluate constant expressions before execution (e.g. `3000 + 2000` → `5000`).
- **Predicate Simplification** — e.g. `salary > 5000 AND salary > 3000` becomes `salary > 5000`.
- **Join Reordering** — chooses a more efficient join order for multi-table joins.

4. Physical Plan

Spark decides *how* the query will run — choosing algorithms such as Broadcast Hash Join, Sort Merge Join, Shuffle Hash Join, Hash Aggregate, Sort Aggregate, File Scan, Exchange (Shuffle). This is the actual execution plan used by the executors.

Full Example

```
employees.filter("salary > 5000") \
  .groupBy("department") \
  .count()
```

Logical Plan:
GroupBy department

Filter salary>5000

Scan employees

Optimized Logical Plan:
Read employees

Push Filter salary>5000

Aggregate department

Physical Plan:
HashAggregate

Exchange (Shuffle)

HashAggregate → Filter → FileScan parquet

The Physical Plan converts into a DAG → Stages → Tasks → executed by Executors, one task per partition.

Viewing the Query Plan

```
df.explain()           # Shows the physical execution plan
```

```
df.explain(True)       # Shows all four stages:
                        # Parsed / Analyzed / Optimized Logical Plan + Physical Plan
```

```
== Parsed Logical Plan ==
Aggregate [department]
+- Filter salary > 5000
```

```
== Analyzed Logical Plan ==
department: string, count: bigint
```

```
== Optimized Logical Plan ==
Aggregate
+- Filter salary > 5000
   +- FileScan parquet
```

```
== Physical Plan ==
HashAggregate
+- Exchange
   +- HashAggregate
```

+ - Filter
+ - FileScan parquet

Summary Table

Stage	Purpose
Logical Plan	Records what operations should be performed
Analyzed Logical Plan	Validates tables, columns, functions, and data types
Optimized Logical Plan	Catalyst Optimizer improves the query (filter pushdown, column pruning, constant folding, join optimization...)
Physical Plan	Chooses execution algorithms and converts the plan into a DAG of stages and tasks

INTERVIEW ANSWER (SHORT)

A Spark Query Plan is the execution blueprint Spark creates for a DataFrame or SQL query. It goes through four stages: Logical Plan, Analyzed Logical Plan, Optimized Logical Plan, and Physical Plan. The Catalyst Optimizer improves the logical plan (filter pushdown, column pruning, constant folding), then Spark generates a physical plan, converts it into a DAG of stages and tasks, and executes those tasks on executors.

What Is a DAG?

A DAG (Directed Acyclic Graph) is Spark's execution plan representing all the transformations required to process data before any computation is executed. Spark builds it automatically, optimizes it, and executes it efficiently when an action is called.

What Does DAG Stand For?

- **Directed** — operations have a specific execution order (parent → child)
- **Acyclic** — no loops or cycles; an operation cannot depend on itself
- **Graph** — a network of operations (nodes) connected by dependencies (edges)

Why Does Spark Use a DAG?

To optimize query execution, minimize unnecessary computations, reduce data shuffling, identify operations that can run in parallel, and improve overall performance.

How Does a DAG Work?

```
df = spark.read.csv("employees.csv", header=True)
result = (
    df.filter(df.salary > 5000)
      .select("department", "salary")
      .groupBy("department")
      .avg("salary")
)
result.show()
```

Read CSV → Filter → Select → GroupBy → Average → Show (Action)
Only when show() is called does Spark execute the DAG.

DAG and Lazy Evaluation

1. Every transformation is recorded
2. Spark builds the DAG
3. Spark optimizes the DAG
4. An action triggers execution

DAG to Stages

The DAG is not executed directly — Spark first divides it into stages. A new stage is created whenever Spark encounters a shuffle (groupBy(), join(), distinct(), orderBy(), repartition()).

```
df.filter(df.salary > 5000) \
  .groupBy("department") \
```



```
.count() \
.show()
```

DAG: Read → Filter → GroupBy [Shuffle] → Count → Show

Stage 1: Read, Filter [Shuffle]

Stage 2: GroupBy, Count, Show

DAG → Stages → Tasks

After creating stages, Spark divides each stage into tasks — each task processes one partition. E.g. 200 partitions → 200 tasks.

User Code → Transformations → DAG → Stages → Tasks → Executors

Who Creates the DAG?

The **Driver Program** creates the DAG: reads your code, builds and optimizes the logical plan, creates the DAG, splits it into stages, splits stages into tasks, and sends tasks to executors.

Advantages of a DAG

- Improves performance via optimized execution
- Reduces unnecessary computations and minimizes data shuffling
- Enables parallel execution
- Supports fault tolerance through lineage — recovers lost partitions without recomputing the entire dataset

DAG vs. Stage vs. Task

Component	Description
DAG	The complete execution plan consisting of all transformations
Stage	A group of transformations that can execute without a shuffle
Task	The smallest unit of work — processes one data partition

Real-World Example

For 500 GB of sales data: read → filter current year → join with customers → group by country → sum sales. Spark builds a DAG of all transformations, optimizes it, splits it into stages (before/after the join/shuffle), creates tasks based on partitions, and executes tasks in parallel across executors.

Interview Q&A

Q1. What is a DAG in Spark?

Spark's execution plan representing all transformations and their dependencies — built and optimized before execution.

Q2. Who creates the DAG?

The Driver Program, after analyzing the transformations in your application.

Q3. When is the DAG executed?

Only when an action (`show()`, `count()`, `collect()`) is called.

Q4. Why is the DAG called "acyclic"?

It has no cycles — an operation cannot depend on itself, so execution always moves forward.

Q5. Relationship between DAG, Stage, and Task?

A DAG is the complete plan; a Stage is a section of the DAG separated by shuffle boundaries; a Task is the smallest execution unit, processing one partition.

INTERVIEW SUMMARY

- A DAG is Spark's optimized execution plan for processing data.
- The Driver builds it from the application's transformations during lazy evaluation.
- When an action is called, Spark optimizes the DAG, splits it into stages at shuffle boundaries, then divides each stage into tasks (one per partition).
- DAG-based execution enables optimization, reduced computation, minimal shuffles, parallel execution, and lineage-based fault recovery.

What Is Hadoop MapReduce?

Hadoop MapReduce is a distributed programming model and processing framework used to process large datasets across multiple machines in a Hadoop cluster. It divides a large processing job into many smaller tasks, processes them in parallel on different machines, and combines the results. It is one of the core components of the Hadoop ecosystem.

Why Do We Need MapReduce?

Processing a 5 TB log file on a single computer would take too long, consume too much memory, and be limited by a single CPU. Instead, MapReduce splits the file into HDFS blocks, processes each block on different machines in parallel, and combines the intermediate results — enabling Hadoop to process massive datasets efficiently.

MapReduce Workflow — Four Phases

HDFS Input Files → Input Splits → Mapper Tasks → (Key, Value Pairs)
→ Shuffle & Sort → Reducer Tasks → Final Output (HDFS)

1. Input Splitting

The input file is stored in HDFS and divided into input splits (usually one split = one HDFS block), each processed independently by a mapper. E.g. a 512 MB file → four 128 MB splits.

2. Map Phase

The Mapper processes one input split at a time and produces key-value pairs (filtering, parsing, transforming — no aggregation).

```
Input:  Apple Banana Apple Orange Apple
Mapper Output:
(Apple, 1) (Banana, 1) (Apple, 1) (Orange, 1) (Apple, 1)
```

3. Shuffle and Sort Phase

The most important — and most expensive — phase. Hadoop groups values with the same key, transfers data between nodes if necessary, sorts the keys, and sends grouped data to reducers.

```
After Shuffle and Sort:
Apple → [1,1,1]
Banana → [1]
Orange → [1]
```

NOTE

The shuffle phase is often the most expensive part of a MapReduce job because it involves moving data across the network.

4. Reduce Phase

The Reducer processes each grouped key and its list of values, aggregates them, and writes the final results back to HDFS.

```
Apple 3
Banana 1
Orange 1
```

Complete Word Count Example

Input: Apple Banana Apple Orange Apple

Step 1 – Map: Apple→1 Banana→1 Apple→1 Orange→1 Apple→1

Step 2 – Shuffle: Apple→[1,1,1] Banana→[1] Orange→[1]

Step 3 – Reduce: Apple 3 Banana 1 Orange 1

Components of MapReduce

Component	Responsibility
Input Split	Divides the input data into smaller pieces
Mapper	Processes input records and produces intermediate key-value pairs
Shuffle & Sort	Groups values by key and sorts the keys before reduction
Reducer	Aggregates values for each key and produces the final output
HDFS	Stores the input and output data
YARN	Allocates cluster resources and schedules MapReduce tasks

Advantages

Processes very large datasets

Parallel execution

Scales horizontally

Fault tolerance (re-runs failed tasks)

Tight HDFS/YARN integration

Limitations

Disk-based (writes intermediate results)

High latency for iterative workloads

Slower than Spark

More complex programming model

MapReduce vs. Apache Spark

Feature	Hadoop MapReduce	Apache Spark
Processing	Disk-based	Primarily in-memory
Speed	Slower	Much faster for most workloads

Execution Model	Map → Shuffle → Reduce	DAG-based execution
Iterative Algorithms	Slow	Efficient
APIs	MapReduce API	RDD, DataFrame, Dataset, SQL
Best For	Large batch processing	Batch, streaming, ML, graph processing

Interview Q&A

Q1. What is Hadoop MapReduce?

A distributed programming model/framework that processes large datasets by dividing work into Map and Reduce tasks executed in parallel across a cluster.

Q2. Phases of MapReduce?

Input Splitting → Map → Shuffle and Sort → Reduce.

Q3. Role of the Mapper?

Reads input records and converts them into intermediate key-value pairs.

Q4. Why is Shuffle and Sort important?

It groups all values with the same key, sorts the keys, and transfers grouped data to the appropriate reducers — without it, reducers couldn't aggregate correctly.

Q5. Role of the Reducer?

Processes each key and its associated values to produce the final aggregated result.

Q6. Why is Spark generally faster than MapReduce?

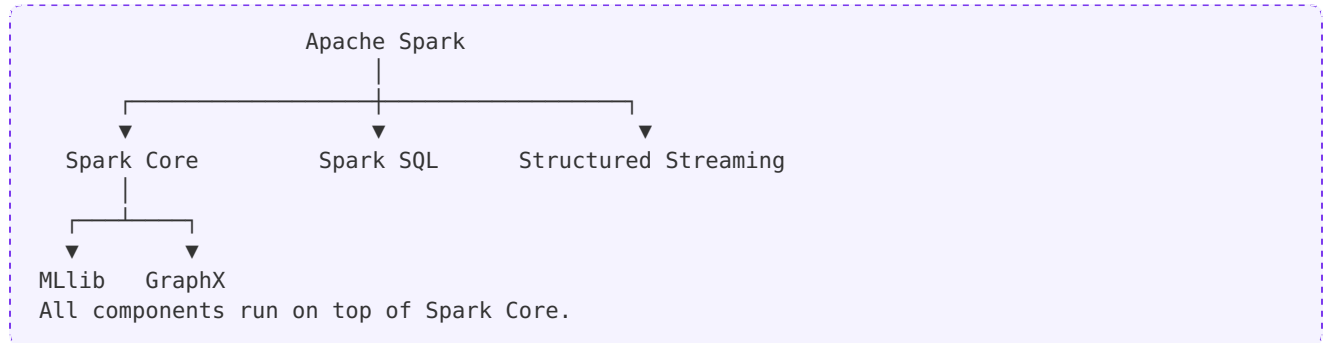
Spark keeps intermediate data in memory whenever possible and uses a DAG-based engine, reducing repeated disk I/O; MapReduce writes intermediate results to disk between Map and Reduce phases.

INTERVIEW SUMMARY

- Hadoop MapReduce is a distributed processing framework for large-scale batch data processing.
- A job consists of Input Splitting, Map, Shuffle and Sort, and Reduce phases.
- The Mapper transforms input into intermediate key-value pairs; Shuffle/Sort groups values by key; the Reducer aggregates and writes final output to HDFS.
- MapReduce is highly scalable and fault tolerant, but generally slower than Spark due to heavy disk I/O.

Apache Spark Components

Apache Spark is a unified analytics engine consisting of several built-in components (libraries), each specialized for a different type of data processing — all running on top of **Spark Core**.



1. Spark Core

The foundation of Apache Spark — every application uses it. Responsible for distributed task execution, memory management, fault tolerance, scheduling, job execution, resource management, I/O, and the RDD API. Without Spark Core, no other component works.

Main features: RDD · DAG Scheduler · Task Scheduler · Fault Recovery via Lineage · in-memory computation · distributed execution.

```
rdd = spark.sparkContext.parallelize([1,2,3,4])    # handled by Spark Core
```

2. Spark SQL

Used for structured and semi-structured data — DataFrames, SQL queries, tables, JSON, CSV, Parquet, ORC, Avro. Uses the Catalyst Optimizer and Tungsten Execution Engine.

Main features: DataFrame API · SQL API · Catalyst Optimizer · cost-based optimization · schema support · column pruning · predicate pushdown.

```
df = spark.read.csv("employees.csv", header=True)
df.filter(df.salary > 5000).show()

# or SQL
df.createOrReplaceTempView("employees")
spark.sql("SELECT * FROM employees WHERE salary > 5000").show()
```

3. Structured Streaming

Spark's stream processing engine — processes continuous data streams using the *same* DataFrame/SQL APIs as batch processing. Sources include Kafka, socket streams, file streams, cloud storage events.

Features: near real-time processing · fault tolerance · exactly-once processing (with supported sinks) · incremental execution · scalable distributed processing.

```
stream = spark.readStream.format("kafka").load()
```

4. MLlib (Machine Learning Library)

Spark's distributed machine learning library — scalable implementations of common ML algorithms that run across a Spark cluster.

Algorithms: Linear/Logistic Regression · Decision Trees · Random Forest · Gradient Boosted Trees · K-Means · ALS Recommendation · PCA · Feature Engineering.

```
from pyspark.ml.classification import LogisticRegression
```

5. GraphX

Spark's graph processing library, for data represented as graphs (vertices/nodes and edges/relationships). Use cases: social networks, fraud detection, recommendation systems, network analysis, route optimization.

NOTE

GraphX is available in Scala and Java only — **not supported in PySpark**. Python users typically use GraphFrames for graph analytics.

How the Components Work Together

1. Read data using Spark SQL
2. Clean and transform it with DataFrames
3. Train a model using MLlib
4. Process live events using Structured Streaming
5. Analyze relationships using GraphX
6. Spark Core manages execution, scheduling, memory, and fault tolerance throughout

Summary Table

Component	Purpose	Main API	Common Use Cases
Spark Core	Core distributed processing engine	RDD	Task scheduling, memory management, fault tolerance
Spark SQL	Structured data processing	DataFrame, SQL	ETL, analytics, SQL queries
Structured Streaming	Real-time stream processing	DataFrame, SQL	Kafka, event processing, live dashboards
MLlib	Distributed machine learning	ML Pipelines	Classification, regression, clustering
GraphX	Graph analytics		

Graph API
(Scala/Java)

Social networks, fraud detection,
recommendations

Interview Q&A

Q1. What are the main components of Apache Spark?

Spark Core, Spark SQL, Structured Streaming, MLlib, GraphX.

Q2. Which component is the foundation of Spark?

Spark Core — all other components rely on it for distributed execution, scheduling, memory management, and fault tolerance.

Q3. Most used component in Data Engineering?

Spark SQL — most workloads involve reading, transforming, joining, aggregating, and writing structured data via DataFrames/SQL.

Q4. Which component processes real-time data?

Structured Streaming — Kafka, sockets, file streams.

Q5. Which component provides ML capabilities?

MLlib — distributed algorithms, feature engineering, model training and prediction.

INTERVIEW SUMMARY

- Spark Core provides distributed processing, memory management, scheduling, and fault tolerance.
- Spark SQL processes structured data with DataFrames and SQL.
- Structured Streaming enables scalable, fault-tolerant real-time processing.
- MLlib provides distributed ML algorithms and pipelines.
- GraphX is Spark's graph library (Scala/Java); Python users typically use GraphFrames.

Job vs. Stage vs. Task — Quick Comparison

A condensed, side-by-side reference for one of the most frequently asked Spark interview questions.

Feature	Job	Stage	Task
Definition	Highest execution unit — entire work triggered by an action	Subset of a job with operations that can run without a shuffle	Smallest execution unit — processes a single partition
Created When	An action runs (show(), count(), collect(), write())	Spark hits a shuffle boundary (groupBy(), join(), orderBy())	Spark creates one task per partition in a stage
Purpose	Executes the complete workflow for the action's result	Organizes the job into phases separated by shuffles	Performs the actual computation on one partition
Granularity	Largest unit	Medium-sized unit	Smallest unit
Contains	One or more stages	One or more tasks	No smaller execution units
Runs On	Managed by the Driver	Scheduled by the Driver	Executed on Executors
Parallelism	Multiple jobs can run concurrently if resources allow	Stages run sequentially within a job; tasks inside run in parallel	Tasks run in parallel across executors
Example	<code>df.filter(...).show()</code> → 1 Job	filter() → Stage 1; groupBy() → Stage 2 (after shuffle)	Stage 1 with 8 partitions → 8 tasks

Execution Hierarchy



Easy Way to Remember

- **Job** → What Spark needs to accomplish (triggered by an action)

- **Stage** → How Spark divides the job into execution phases (separated by shuffles)
- **Task** → Who does the work (one task processes one partition on an executor)

Example

```
df.filter(df.salary > 5000) \  
  .groupBy("department") \  
  .count() \  
  .show()
```

show() → Creates 1 Job
↓
Stage 1 → Read CSV + Filter
↓
[Shuffle]
↓
Stage 2 → GroupBy + Count
↓
Tasks → One task per partition in each stage

MEMORY SHORTCUT

- **Action** ⇒ Job
- **Shuffle** ⇒ New Stage
- **Partition** ⇒ Task

SparkSession vs. SparkContext

What Is SparkSession?

The entry point to all Spark functionality in Spark 2.x and later — a single unified interface for DataFrames, Spark SQL, Structured Streaming, MLlib, and SparkContext (RDD operations). It replaces having to create multiple contexts (SQLContext, HiveContext, SparkContext) separately.

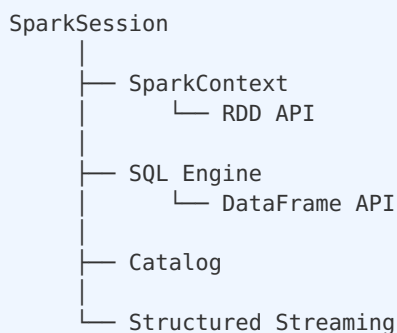
```
from pyspark.sql import SparkSession
spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()
```

What Is SparkContext?

The core engine of Apache Spark — establishes the connection between your application and the Spark cluster, and is responsible for executing Spark jobs. Mainly used for RDD operations.

```
sc = spark.sparkContext
rdd = sc.parallelize([1, 2, 3, 4])
print(rdd.collect())
```

Relationship Between Them



When you create a SparkSession, Spark automatically creates (or reuses) a SparkContext:

```
spark = SparkSession.builder.getOrCreate()
sc = spark.sparkContext
```

KEY POINT

Every SparkSession contains a SparkContext, but a SparkContext does **not** contain a SparkSession.

Responsibilities

SparkSession: entry point for all Spark applications · creates DataFrames · executes SQL queries · reads/writes data · manages configurations · provides access to SparkContext · supports Structured Streaming and MLlib.

SparkContext: connects the application to the cluster · creates RDDs · sends jobs to the cluster manager · schedules tasks · communicates with executors · monitors execution · collects results to the driver.

Key Differences

Feature	SparkSession	SparkContext
Introduced	Spark 2.0	Spark 1.x
Purpose	Unified entry point for all Spark APIs	Core execution engine
Main API	DataFrame, SQL, Streaming, MLlib	RDD
Creates DataFrames	Yes	No
Executes SQL	Yes	No
Creates RDDs	Indirectly (via spark.sparkContext)	Yes
Connects to Cluster	Through SparkContext	Yes
Used in Modern Spark	Yes	Used mainly for RDDs
Contains the Other?	Contains/accesses SparkContext	Does not contain SparkSession

Examples

```
# Using SparkSession (Recommended)
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Demo").getOrCreate()
df = spark.read.csv("employees.csv", header=True)
df.show()

# Using SparkContext (RDD)
sc = spark.sparkContext
numbers = sc.parallelize([1, 2, 3, 4, 5])
squared = numbers.map(lambda x: x * x)
print(squared.collect())
```

Which One Should You Use?

- Use **SparkSession** for almost all modern Spark applications.
- Use **SparkContext** only when you need low-level RDD operations or legacy RDD-based code.

Interview Q&A

Q1. Why was SparkSession introduced?

Before Spark 2.0, developers had to create multiple contexts (SparkContext, SQLContext, HiveContext). SparkSession unified these into a single entry point, simplifying Spark application development.

Q2. Does SparkSession replace SparkContext?

No — it wraps and manages it. Creating a SparkSession automatically creates or reuses a SparkContext internally.

Q3. Can you create an RDD using SparkSession?

Not directly — access the underlying SparkContext:

```
spark.sparkContext.parallelize([1,2,3])
```

Q4. Can SparkContext execute SQL queries?

No — SparkContext only supports RDD operations; SQL queries require SparkSession.

Q5. Which is the entry point of Spark?

Spark 1.x → SparkContext. Spark 2.x and later → SparkSession.

INTERVIEW SUMMARY

- SparkSession is the modern, unified entry point to Apache Spark — supports DataFrames, Spark SQL, Structured Streaming, MLlib, and provides access to SparkContext.
- SparkContext is the core execution engine connecting the application to the cluster, primarily responsible for RDD operations, job scheduling, and task execution.
- In modern Spark applications: create a SparkSession first, and reach for SparkContext only when RDD-specific functionality is needed.